

Desenvolvimento Web II

Professor Allyson Souza

Desenvolvimento Web 2 © 2026 por Allyson Souza está licenciado sob a **CC BY-NC 4.0**. Para ver uma cópia da licença, visite <https://creativecommons.org/licenses/by-nc/4.0/>

A large yellow square containing the letters 'JS' in a bold, dark gray, sans-serif font, representing JavaScript.

Introdução à linguagem

Introdução ao JavaScript

O JavaScript foi criado inicialmente para “tornar as páginas da web vivas”.

1. Os programas nesta linguagem são chamados de scripts. Eles podem ser escritos diretamente no HTML de uma página da web e executados automaticamente conforme a página carrega.
2. Os scripts são fornecidos e executados como texto simples. Eles não precisam de preparação ou compilação especial para serem executados.
3. Nesse aspecto, o **JavaScript** é muito diferente de outra linguagem chamada **Java**.

Introdução ao JavaScript



Por que a linguagem se chama JavaScript?

Quando o JavaScript foi criado, ele tinha inicialmente outro nome: “LiveScript”. Mas o Java era muito popular naquela época, então decidiu-se que posicionar uma nova linguagem como o “irmão mais novo” do Java seria vantajoso.

O JavaScript é uma linguagem totalmente independente, com sua própria especificação chamada **ECMAScript**, e não tem nenhuma relação com o Java para além dessa curiosidade.

Ambiente de execução

Hoje, o JavaScript pode ser executado não apenas no navegador, mas também no servidor ou, na verdade, em qualquer dispositivo que possua um programa especial chamado de **JavaScript Engine**.

O navegador possui um motor embutido, chamado de “máquina virtual de JavaScript”.

Motores diferentes têm nomes diferentes. Os “Três Grandes” motores modernos são:

- **V8** – nos navegadores Chrome, Edge, Opera e Brave, e também no Node e Deno
- **SpiderMonkey** – no Firefox.
- **JavaScriptCore** – no Safari e no Bun.

É bom lembrar os termos acima, pois eles são usados em artigos para desenvolvedores na internet. Nós também vamos usá-los.

Introdução ao JavaScript



Como funcionam os motores?

Os motores são complexos. Mas os princípios básicos são simples.

1. O motor (embutido, se for um navegador) lê (“analisa”) o script.
2. Em seguida, ele converte (“compila”) o script em código de máquina.
3. E então o código de máquina é executado, com bastante rapidez.

O motor aplica otimizações em cada etapa do processo. Ele até monitora o script compilado enquanto ele é executado, analisa os dados que passam por ele e otimiza ainda mais o código de máquina com base nesse conhecimento.

O que JavaScript pode fazer?

O JavaScript moderno é uma linguagem de programação "segura". Ele não oferece acesso de baixo nível à memória (RAM) ou à CPU, pois foi criado inicialmente para navegadores, que não exigem isso.

Os recursos do JavaScript dependem muito do ambiente em que está sendo executado.

JavaScript no navegador

O JavaScript no navegador pode realizar todas as tarefas relacionadas à manipulação de páginas da web, à interação com o usuário e ao servidor web. Por exemplo, o JavaScript no navegador é capaz de:

- Adicionar novo código HTML à página, alterar o conteúdo existente e modificar estilos.
- Reagir às ações do usuário, ser executado ao clicar com o mouse, ao mover o ponteiro ou ao pressionar teclas.
- Enviar solicitações pela rede para servidores remotos, baixar e enviar arquivos.
- Obter e definir cookies, fazer perguntas ao visitante, exibir mensagens.
- Lembrar os dados no lado do cliente (“armazenamento local”).

JavaScript no navegador: o que não é permitido

As capacidades do JavaScript no navegador são limitadas para proteger a segurança do usuário. O objetivo é impedir que uma página da web maliciosa acesse informações privadas ou prejudique os dados do usuário.

- O JavaScript em uma página da web não pode ler/gravar arquivos arbitrários no disco rígido, copiá-los ou executar programas. Ele não tem acesso direto às funções do sistema operacional.

JavaScript no navegador: o que não é permitido

#1 Leitura e escrita de arquivos

O JavaScript em uma página da web não pode ler/gravar arquivos arbitrários no disco rígido, copiá-los ou executar programas. Ele não tem acesso direto às funções do sistema operacional.

JavaScript no navegador: o que não é permitido

#2 "Same Origin Policy" - Política da mesma origem

Guias/janelas diferentes geralmente não têm conhecimento umas das outras.

Às vezes têm, por exemplo, quando uma janela usa JavaScript para abrir outra. Mas mesmo nesse caso, o JavaScript de uma página não pode acessar a outra página se elas forem de sites diferentes (de um domínio, protocolo ou porta diferentes).

Isso é chamado de “Política de Mesma Origem”. Para contornar isso, ambas as páginas devem concordar com a troca de dados e conter um código JavaScript especial que faça essa gestão.

O que faz do JavaScript uma linguagem única?

- Integração total com HTML e CSS
- Sintaxe simples e versátil
- Suportada por todos os principais navegadores e habilitada por padrão

Linguagens que transpilam para JavaScript

Diferentes pessoas e projetos possuem diferentes necessidades, então o JavaScript não agrada a todos. Por isso hoje existem uma série de linguagens que são *transpiladas*, ou seja, possuem código semelhante, mas não igual, e que compilam para JavaScript.

Algumas delas: **TypeScript, Dart, Kotlin, ClojureScript**

Resumo

- JavaScript foi inicialmente projetada como uma linguagem específica para navegadores, agora ela é utilizado em muitos outros ambientes de execução.
- Hoje, o JavaScript tem uma posição única de linguagem padrão, completamente integrada à HTML e CSS nos navegadores.
- No navegador, as permissões do que podemos fazer com a linguagem JavaScript são mais restritas e controladas, garantindo maior segurança.

Estrutura léxica

Introdução

A estrutura léxica de uma linguagem de programação é o conjunto de regras elementares que especifica como se escrevem programas nessa linguagem.

Trata-se da sintaxe de nível mais baixo de uma linguagem: ela especifica, por exemplo, como são os nomes das variáveis, os caracteres delimitadores dos comentários e como uma instrução do programa é separada da seguinte.

Maiúsculas e minúsculas

JavaScript é uma linguagem que distingue maiúsculas e minúsculas. Isso significa que as palavras chave da linguagem, variáveis, nomes de funções e outros identificadores devem sempre ser digitados de maneira consistente quanto a este aspecto.

A palavra chave **while**, por exemplo, deve ser digitada **while**, não **While** ou **WHILE**.

O mesmo é válido para nomes de variáveis. Um variável chamada **online** precisa ser sempre referenciada dessa forma, **Online** ou **ONLINE** serão interpretados como variáveis diferentes.

Espaços

O JavaScript ignora os espaços que aparecem entre os tokens nos programas. Na maioria dos casos, o JavaScript também ignora as quebras de linha na maioria dos casos.

Como é possível usar espaços e quebras de linha livremente em seus programas, você pode formatar e recuar seus programas de maneira organizada e consistente, tornando o código fácil de ler e compreender.

Comentários

O JavaScript suporta dois tipos de comentários.

1. Qualquer texto entre `//` e o final de uma linha é tratado como um comentário e ignorado pelo JavaScript.
2. Qualquer texto entre os caracteres `/*` e `*/` também é tratado como um comentário; esses comentários podem se estender por várias linhas.

```
// Este é um comentário de uma linha.
```

```
/* Este também é um comentário */ // e aqui está outro comentário.
```

```
/*  
 * Este é um comentário de várias linhas. Os caracteres  
 * extras no início de  
 * cada linha não são obrigatórios na sintaxe; eles só  
 * ficam legais!  
 */
```

Literais

Literal é qualquer valor que aparece diretamente no programa. Os valores ao lado são todos literais.

`12` // 0 número doze

`1.2` // 0 número um ponto dois

`"olá mundo"` // Uma sequência de texto

`'Oi'` // Outra sequência de texto

`true` // Um valor booleano

`false` // 0 outro valor booleano

`null` // Ausência de um objeto

Identificadores e palavras reservadas

Um identificador é simplesmente um nome.

No JavaScript, os identificadores são usados para nomear **constantes**, **variáveis**, **propriedades**, **funções** e **classes**.

Um identificador em JavaScript **deve** começar com uma letra, um sublinhado (`_`) ou um cifrão (`$`). Os caracteres seguintes podem ser letras, dígitos, sublinhados ou cifrões.

(Não é permitido usar dígitos como primeiro caractere, para que o JavaScript possa distinguir facilmente identificadores de números)

```
// Identificadores válidos
```

```
i
```

```
minha_variável
```

```
v13
```

```
__carro
```

```
$string
```

```
// Identificadores inválidos
```

```
1pessoa
```

```
#carro
```

```
lista*de*compras
```

```
>alvo<
```

Palavras reservadas

As palavras reservadas são parte da linguagem. Elas não devem ser usadas como nomes de variáveis, constantes, funções ou classes.

as	const	export	get	null	target	void
async	continue	extends	if	of	this	while
await	debugger	false	import	return	throw	with
break	default	finally	in	set	true	yield
case	delete	for	instanceof	static	try	
catch	do	from	let	super	typeof	
class	else	function	new	switch	var	

Palavras reservadas

Algumas palavras reservadas ainda não são usadas pela linguagem, mas podem vir a ser, por isso também são restritas.

enum	implements	interface	package	private	protected	public
------	------------	-----------	---------	---------	-----------	--------

Ponto e vírgula

Como muitas outras linguagens, JavaScript também utiliza ponto e vírgula para separar instruções. Isso é importante para tornar o seu código compreensível.

Sem o ponto e vírgula, o final de uma instrução pode ser confundido com o início de outra.

// Ponto e vírgula opcional

```
a = 3;
```

```
b = 4;
```

```
c = 5;
```

// Ponto e vírgula obrigatório

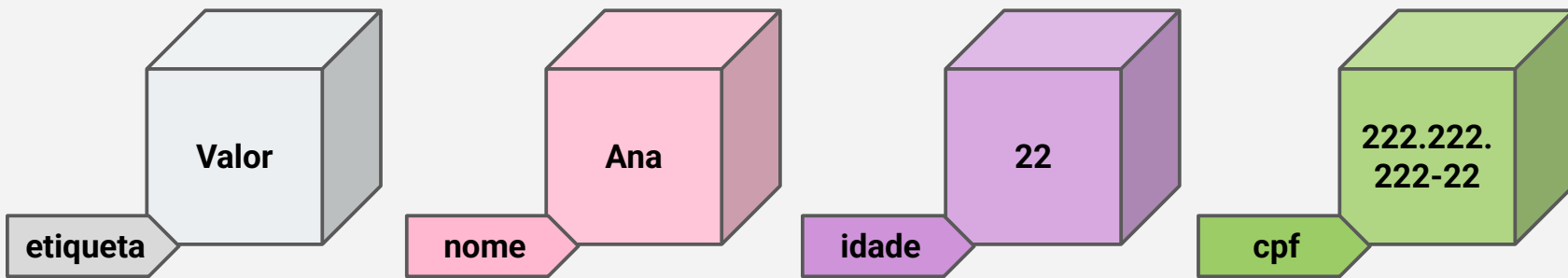
```
a = 3; b = 4; c = 5;
```

Tipos, valores e variáveis

O que são variáveis?

Podemos pensar em variáveis, metaforicamente, como *caixas etiquetadas*, em que iremos armazenar alguma coisa. No caso de linguagens de programação, iremos armazenar dados.

Por exemplo, podemos criar uma variável para armazenar um nome, ou uma idade, ou um CPF, por exemplo.



Declaração de variáveis

Para declararmos variáveis no JavaScript, usamos a palavra chave **let** seguida do nome que queremos para nossa variável (obedecendo a regra de nomeação de identificadores da linguagem).

*"Declarar uma variável é **reservar um espaço** na memória RAM do computador, para **armazenar um valor**, que pode ser resgatado futuramente usando o nome o qual atribuímos."*

```
// Declaração da variável
```

```
let mensagem;
```

```
// Atribuição de um valor à variável
```

```
mensagem = 'Olá!';
```

```
// Usando o valor da variável
```

```
alert(mensagem);
```

Declaração de variáveis: múltiplas por linha

Podemos declarar múltiplas variáveis por linha, separando seus identificadores por vírgula, ou mesmo usando uma instrução `let`, separando por vírgulas e quebrando linhas

Não é muito recomendado, dado o prejuízo quanto à legibilidade do código.

```
let usuario = 'John', idade = 25;
```

```
let usuario = 'John',  
    idade = 25;
```

Tipos de dados

Existem alguns tipos de dados que podemos armazenar em JavaScript.

O JavaScript possui ***tipagem dinâmica***, o que significa que você não precisa dizer ao navegador qual é o tipo do dado; ele descobre sozinho em tempo de execução, ou seja, enquanto o código é interpretado por ele - diferente de algumas outras linguagens.

Strings

Representa dados textuais, como nomes, códigos alfanuméricos, mensagens, etc.

Uma String deve sempre estar envolvida por aspas simples (' '), aspas duplas (" ") ou crases (` `).

```
let nome = "José"
```

```
let rua = "R. das Flores, 123"
```

```
let cpf = "222.222.222-22"
```

Strings

As crases tem uma “funcionalidade estendida”.

Elas nos permitem incorporar variáveis e expressões em uma string, colocando-as entre `${...}`, por exemplo:

```
let name = "Jessica";
```

```
// Usando crases para incorporar a  
variável
```

```
alert( `Olá, ${name}!` );
```

```
// embed an expression
```

```
alert( `the result is ${1 + 2}` ); //  
the result is 3
```

Números

Representa dados numéricos.

Diferente de outras linguagens (que separam inteiros e decimais), o JavaScript usa o tipo Number para tudo: números inteiros (10), decimais (3.14) ou negativos (-5).

```
let idade = 27
```

```
let saldo = 110.57
```

```
let temperatura = -2
```

Números: valores numéricos especiais

Para além dos números, existem os chamados valores numéricos especiais, que são:

- **Infinity**
- **-Infinity**
- **NaN**

Os valores numéricos especiais pertencem formalmente ao tipo “número”. É claro que não são números no sentido comum da palavra.

```
// Divisão por zero
```

```
alert( 1 / 0 ); // Infinity
```

```
// Multiplicação com string
```

```
alert( "A" * 2 ); // NaN
```

Booleano

Pode ter apenas um de dois valores: **true** (verdadeiro) ou **false** (falso).

É fundamental para tomar decisões lógicas no código (estruturas como o if).

```
let maiorDeIdade = true
```

```
let matriculado = true
```

```
let ligado = false
```

Undefined

É o valor padrão atribuído pelo JavaScript a uma variável que foi declarada, mas que ainda não recebeu nenhum valor.

Todas as variáveis ao lado, a princípio, terão armazenadas o valor **undefined** (indefinido).

```
let nome
```

```
let idade
```

```
let endereco
```

Null

É a ausência *intencional* de qualquer valor.

Diferente do undefined, o null é atribuído pelo programador quando ele quer dizer explicitamente que "esta variável está vazia".

```
let nome = null
```

```
let idade = null
```

```
let endereco = null
```

Operador typeof

Muitas vezes não sabemos bem, ou iremos querer nos certificar de que uma variável está armazenando um determinado tipo de dados. Para isso, podemos utilizar o operador typeof.

Uso do typeof

```
let nomeDoAluno = "Marcos";  
console.log("Valor:", nomeDoAluno);  
console.log("Tipo:", typeof nomeDoAluno);
```

Declaração com let e const

Declaração com **let**

No JavaScript atual (ES6 em diante), as variáveis são declaradas com a instrução **let**.

- É uma boa prática de programação atribuir valores ao declarar variáveis.
- Podemos utilizar as variáveis previamente declaradas na atribuição de valor das novas.

```
// Uma variável por linha
```

```
let i;
```

```
let sum;
```

```
// Múltiplas variáveis por linha
```

```
let i, sum;
```

```
// Atribuição de valor ao declarar
```

```
let message = "hello";
```

```
let i = 0, j = 0, k = 0;
```

```
let x = 2, y = x*x;
```

Declaração com **const**

Utilizando a instrução **const**, podemos declarar variáveis que não podem ter seu valor redefinido.

```
const PI = 3.14159;  
  
const MAXIMO_TENTATIVAS_LOGIN = 5;  
  
const API_URL = "https://example.com";
```

Existem duas correntes de pensamento sobre o uso da palavra-chave `const`.

Uma abordagem consiste em usar `const` apenas para valores que são fundamentalmente imutáveis, como as constantes físicas mostradas, ou números de versão de programas, ou sequências de bytes usadas para identificar tipos de arquivos, por exemplo.

Outra abordagem reconhece que muitas das chamadas “variáveis” em nosso programa, na verdade, nunca mudam enquanto o programa está em execução. Nessa abordagem, declaramos tudo com “`const`” e, se percebermos que realmente queremos permitir que o valor varia, alteramos a declaração para “`let`”. Isso pode ajudar a evitar bugs ao impedir alterações acidentais em variáveis que não pretendíamos.

Em uma abordagem, usamos `const` apenas para valores que não devem mudar. Na outra, usamos `const` para qualquer valor que por acaso não mude. Eu prefiro a primeira abordagem no meu próprio código.

David Flanagan, **JavaScript: The Definitive Guide**, p. 72

Incluindo JavaScript em uma página HTML

```
<script></script>
```

Tag <script>

Para adicionarmos JavaScript à uma página da web, precisamos de uma tag específica que diz que iremos vincular um código JavaScript à nossa página.

```
<script>
// Posso escrever JavaScript
// dentro da tag script.
// Chamamos isso de script incorporado.
</script>
```

```
<!-- Chamada de script externo -->
<script src="meuscript.js"></script>
```

Interação básica

`alert()` / `prompt()` / `confirm()` / `console`

alert()

A instrução alert() exibe uma mensagem em uma janela de diálogo (pop-up) do navegador.

Essa mensagem precisa ser passada entre aspas, dentro dos parênteses.

```
alert("Estou aprendendo JavaScript!");
```

confirm()

A função **confirm** exibe uma janela modal com uma pergunta e dois botões: **OK** e **Cancelar**.

O resultado é **true** se o botão OK for pressionado e **false** caso contrário.

```
let aluno
```

```
aluno = confirm("Você é um aluno?");
```

```
alert( aluno ); // true ou false
```

prompt()

Exibe uma janela modal com uma mensagem de texto, um campo de entrada para o visitante e os botões OK/Cancelar.

título

O texto a ser exibido ao visitante.

padrão

Um segundo parâmetro opcional, que representa o valor inicial do campo de entrada.

```
let idade;
```

```
idade = prompt('Qual sua idade?', 100);
```

```
alert(`Você tem ${age} anos!`);
```

```
// Saída: Você tem __ anos!
```

console.log()

O `console.log()` é a ferramenta padrão de diagnóstico do desenvolvedor.

Ele não exibe nada na tela para o usuário final; em vez disso, ele envia o dado diretamente para o painel de controle interno do navegador (as chamadas DevTools).

```
let nome = "Marcos";  
  
console.log("Valor:", nome);
```

Conversão de tipos

Conversão de tipos

Na maioria das vezes, os operadores e as funções convertem automaticamente os valores que lhes são fornecidos para o tipo correto.

Por exemplo, a função **alert()** converte automaticamente qualquer valor em uma string para exibi-lo.

As operações matemáticas convertem valores em números.

Conversão de tipos

`10 + " objetos" // "10 objetos"` número 10 é convertido em string

`"7" * "4" // => 28`, ambas as strings são convertidas para números

`let n = 1 - "x"; // n == NaN`; string "x" can't convert to a number

`n + " objects" // "NaN objects"`: NaN é convertido para string "NaN"

Tabela de conversões

parte 1

Resume como os valores são convertidos de um tipo para outro no JavaScript.

As entradas em negrito na tabela destacam conversões que podem te surpreender. As células vazias indicam que nenhuma conversão é necessária e, portanto, nenhuma é realizada.

Valor	to String	to Number	to Boolean
undefined	"undefined"	NaN	false
null	"null"	0	false
true	"true"	1	true
false	"false"	0	false
" " (empty string)	""	0	false
"1.2" (nonempty, numeric)	"1.2"	1.2	true
"one" (nonempty, non-numeric)	"one"	NaN	true
0	"0"		false
-0	"0"		false
1 (finito, não-zero)	"1"		true

Tabela de conversões

parte 2

Resume como os valores são convertidos de um tipo para outro no JavaScript.

As entradas em **negrito** na tabela destacam conversões que podem te surpreender. As células vazias indicam que nenhuma conversão é necessária e, portanto, nenhuma é realizada.

Valor	to String	to Number	to Boolean
Infinity	"Infinity"		true
-Infinity	"-Infinity"		true
NaN	"NaN"		false
{ } qualquer objeto	<i>a detalhar mais adiante</i>	<i>a detalhar mais adiante</i>	true
[] array vazio	""	0	true
[9]	"9"	9	false
['a']	precisa do método join()	NaN	true
function() {}	<i>a detalhar mais adiante</i>	NaN	true

Conversões e igualdades

Conversões e igualdade

O JavaScript possui dois operadores que verificam se dois valores são iguais.

O “operador de igualdade estrita”, `===`, não considera seus operandos iguais se eles não forem do mesmo tipo, e esse é quase sempre o operador correto a ser usado na programação.

Mas, como o JavaScript é tão flexível em relação às conversões de tipo, ele também define o operador `==` com uma definição flexível de igualdade.

Todas as comparações a seguir são verdadeiras, por exemplo:

Conversões e igualdade

`null == undefined` // => true: estes dois valores são tratados como iguais

`"0" == 0` // => true: String converte para número antes da comparação

`0 == false` // => true: Boolean converte para número antes da comparação

`"0" == false` // => true: Ambos operandos são convertidos para 0 antes da comparação

Conversões explícitas

Conversão de strings

A conversão de string ocorre quando precisamos da forma de string de um valor.

Por exemplo, a função **alert(valor)** faz isso para exibir o valor.

Também podemos chamar a função **String(valor)** para converter um valor em uma string:

```
let valor = true;
```

```
alert(typeof valor); // boolean
```

```
valor = String(valor); // agora value  
is a string "true"
```

```
alert(typeof value); // string
```

Conversão de números

A conversão numérica em funções e expressões matemáticas ocorre automaticamente.

Por exemplo, quando a divisão / é aplicada a valores que não são números:

```
alert( "6" / "2" );  
// Saída: 3, as strings são  
// automaticamente convertidas em  
// números
```

Conversão de números

Podemos usar a função **Number(valor)** para converter explicitamente um valor em um número:

```
let palavra = "123";  
alert(typeof palavra); // string
```

```
let num = Number(palavra);  
// transforma no número 123
```

```
alert(typeof num); // number
```

Conversão de números

Podemos usar a função **Number(valor)** para converter explicitamente um valor em um número:

```
let palavra = "123";  
alert(typeof palavra); // string
```

```
let num = Number(palavra);  
// transforma no número 123
```

```
alert(typeof num); // number
```

Operadores aritméticos

Operadores aritméticos

```
console.log(5 + 7)
```

```
console.log(10 - 4)
```

```
console.log(2 * 8)
```

```
console.log(34 / 17)
```

```
console.log(3 ^ 2)
```

Condicionais

`if / else / else if`

Analogia do cotidiano: o semáforo

Imagine que você está chegando a um cruzamento. Você precisa tomar uma decisão baseada em uma condição (a cor do sinal):

SE o sinal estiver verde, **ENTÃO** você atravessa a rua.

SENÃO (se estiver vermelho ou amarelo), **ENTÃO** você espera na calçada.

Na programação, o **se** funciona exatamente assim: ele testa uma condição. Se ela for verdadeira, o programa executa uma ação; se for falsa, ele executa outra (o senão).

A estrutura das condições em programação

Podemos dividir a estrutura de decisão em três partes:

A **Condição**: Uma pergunta que só pode ter duas respostas: Verdadeiro ou Falso (Ex: idade $\geq$ 18).

O **Bloco Verdadeiro** (se, ou **if**, em inglês): O que o computador deve fazer se a resposta for "Sim".

O **Bloco Falso** (senão, ou **else**, em inglês): O que o computador deve fazer se a resposta for "Não".

Estrutura da instrução if...else

Veja a estrutura visual da instrução se...senao ao lado. Observe que os caracteres de chaves { } são utilizados para delimitar os blocos de código em Portugal (e em muitas outras linguagens de programação, mas não em todas)

condição

```
if( idade >= 18 ) {  
    console.log("É maior de idade.")  
}  
else {  
    console.log("É menor de idade.")  
}
```

Bloco se verdadeiro

Bloco se Falso

Comparando variáveis e valores

Como podemos ver no exemplo anterior, **como comparar se uma idade, armazenada em uma variável, é maior ou menor que um determinado valor?**

Ou como poderíamos comparar o valor entre duas variáveis, para saber se são iguais?

As linguagens de programação nos fornecem então os **operadores de comparação** para isso.

Tabela de operadores relacionais

Operador	Significado	Exemplo	Resultado
===	Igual e do mesmo tipo de	5 === "5"	false
==	Igual a	5 == 5	true
!=	Diferente de	5 != 3	true
>	Maior que	10 > 20	false
<	Menor que	7 < 15	true
>=	Maior ou igual a	10 >= 10	true
<=	Menor ou igual a	12 <= 10	false

Atenção

Muitos iniciantes confundem o **operador de atribuição** com o de **comparação de igualdade**.

= (Um sinal): Atribuição

Você está "guardando" um valor dentro de uma caixa.

Ex: idade = 20 (A idade agora é 20).

== (Dois sinais): Comparação de igualdade

Você está "perguntando" se os valores são iguais.

Ex: idade == 20? (A idade é igual a 20?).

Encadeamento

Após a verificação de uma condição if, em um bloco else, posso segui-lo de uma nova verificação, antes de executar diretamente o bloco. Esse encadeamento de condições é o que chamamos de **else if** (senão se...)

```
condição 1      condição 2
┌──────────┐
if( idade >= 60 ) {
    console.log("É idoso.")
} else if ( idade >= 18 ) {
    console.log("É maior de idade.")
} else {
    console.log("É menor de idade.")
}
```

Condicionais compostas

&& / ||

Operadores lógicos

Muitas vezes precisamos fazer comparações de mais de uma condição de uma vez, como vimos nos exercícios de tabelas verdades.

Conseguimos utilizar estas condicionais compostas em instruções **se**, por exemplo.

Tabela de operadores lógicos

Operador	Significado	Exemplo (Portugol)	Resultado do exemplo
&&	Verdadeiro se as duas condições são verdadeiras	5 > 4 && 17 <= 10	falso
 	Verdadeiro se ao menos uma das condições for verdadeira	10 > 11 10 > 5	verdadeiro
!	Inverte o valor lógico de uma condição	! true	falso

Caso

switch ... case

O que é *switch...case*

O *switch case* é uma estrutura de controle que avalia uma expressão e executa um bloco de código correspondente ao valor encontrado.

É uma alternativa mais legível ao encadeamento de vários *if/else*.

Sintaxe básica

Essa é a sintaxe básica da estrutura switch case

```
switch (expressão) {  
    case valor1:  
        // executado se expressão == valor1  
        break;  
    case valor2:  
        // executado se expressão == valor2  
        break;  
    default:  
        // executado se nenhum case corresponder  
}
```

Exemplo de uso

Queremos testar se a variável `cor` possui um dos seguintes valores atribuído a ela.

```
let cor = "Amarelo"

switch(cor){

  case "Amarelo":

    console.log("Amarelo: é uma cor primária")

    break;

  case "Azul":

    console.log("Azul: é uma cor primária")

    break;

  default:

    console.log("Não é uma cor primária")

}
```

Funções

declaração, propriedades e manipulação

O que são funções?

As funções são um dos recursos mais importantes na programação JavaScript.

O conceito de envolver uma parte de um programa em um valor tem muitos usos. Ele nos permite estruturar programas maiores, reduzir repetições, associar nomes a subprogramas e isolar esses subprogramas uns dos outros.

3 (três) formas de criação de funções

O JavaScript possui 3 (três) principais formas de criação de funções, que são:

1. **Declaração de funções** (ou funções nomeadas)
2. **Expressão de funções** (atribuindo à variáveis, constantes e propriedades de objetos)
3. **Funções seta** (*arrow functions*)

Iremos cobrir as 2 (duas) primeiras a seguir.

1. Atribuindo uma função

Uma atribuição de função é uma declaração de uma variável ou constante em que o valor associado a este identificador é uma função.

Por exemplo, este código define a constante `quadrado` para se referir a uma função que produz o quadrado de um determinado número.

```
const quadrado = function (x) {  
    return x * x;  
};
```

// Invocação da função `quadrado`
passando o argumento `12`

```
console.log(quadrado(12));
```

// Saída → 144

Constante que irá armazenar a função

Instrução function

Parâmetro

```
1 const quadrado = function(x) {  
2     return x * x;  
3 }
```

Retorno

Estrutura de atribuição de uma função

Parâmetros

Uma função pode ter **vários parâmetros** ou **nenhum parâmetro**.

No exemplo a seguir, fazerBarulho não lista nenhum nome de parâmetro, enquanto soma (que faz a soma de dois números) recebe dois parâmetros:

```
const fazerBarulho = function() {  
  console.log("Vruuum!")  
}
```

```
fazerBarulho()  
// → Vruuum!
```

```
const soma = function(a, b) {  
  let resultado = a + b  
  return resultado  
};
```

```
console.log(soma(23, 5))  
// → 28
```

2. Declaração de função

Existe uma maneira um pouco mais curta de criar uma função.

Quando a palavra-chave ***function*** é usada no início de uma instrução, ela funciona de maneira diferente

```
function quadrado(x) {  
    return x * x;  
}
```

Declaração de funções / função nomeada

```
function areaDoRetangulo(ladoA, ladoB) {  
    let area = ladoA * ladoB  
    return area  
}
```

// Invocação da função com diferentes parâmetros

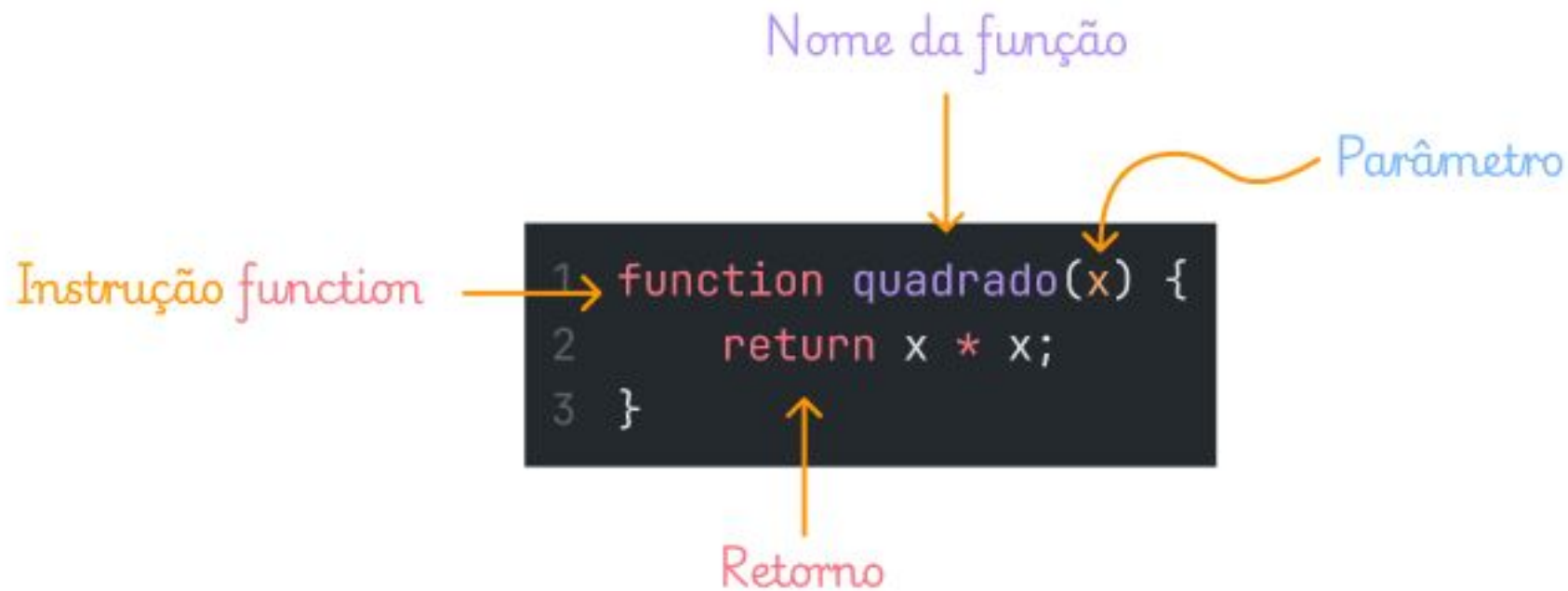
```
areaDoRetangulo(10, 10)
```

```
areaDoRetangulo(2, 4)
```

```
areaDoRetangulo(5, 3)
```

Declaração de funções / função nomeada

```
function areaDoRetangulo(ladoA, ladoB) {  
    let area = ladoA * ladoB  
    return area  
}
```



Estrutura de declaração de uma função

Detalhes de declaração de funções

Há uma sutileza nessa forma de definição de função.

O código anterior funciona, mesmo que a função seja definida abaixo do código que a utiliza.

As declarações de função não fazem parte do fluxo de controle regular de cima para baixo.

Elas são conceitualmente movidas para o topo de seu escopo e podem ser utilizadas por todo o código nesse escopo. Isso às vezes é útil porque oferece a liberdade de ordenar o código da maneira que parecer mais clara, sem se preocupar em definir todas as funções antes de utilizá-las.

```
console.log(  
    "Mensagem do futuro: ",  
    mensagemDoFuturo()  
)  
  
function mensagemDoFuturo() {  
    return "A IA ainda não capina  
lote";  
}
```

Closures

As definições de funções em JavaScript podem ser *aninhadas* dentro de outras funções e têm acesso a todas as variáveis que estão no escopo em que são definidas.

Isso significa que as funções em JavaScript são ***closures***, o que possibilita técnicas de programação importantes e poderosas.